

Syntax and Parsing

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

meaning 需要 structure: 句子不是词袋, 谁修饰谁、谁和谁成一组, 会直接改变语义 → syntax as shortcut: 树结构把线性文本里相隔很远的词变成直接相连的节点 → two views of trees: constituency 看短语盒子, dependency 看词与词的 head-dependent 关系 → grammar 的理想: 有限规则、能解释语义、还能捕捉语言的结构与分布规律 → parsing task: 给一句话, 输出对应的 syntactic tree; 可监督也可无监督, 本课重点是 treebank-supervised → early neural move: 把 tree linearise 成 bracket sequence, 用 seq2seq 像翻译一样生成树 → evaluation: 不只问整棵树是否完全一样, 而是比较 constituents 的 span、label、precision、recall 和 F1 → Transformer parser: 编码每个词的上下文, 再给每个 span-label 打分 → global decoding: 局部 span 分数不能随便堆, 需要 CYK/dynamic programming 找合法最高分 tree → attention details: factored attention 分离 content 与 position, 让模型更适合处理 span 和层级结构

Syntax

- Syntactic understanding of text was needed as a scaffold for understanding meaning
- Understanding the syntactic structure of text, in a sense, “shortcuts” distances between words and phrases in (otherwise sequential) text, providing direct relationship between them that can be further processed
- LLMs taught us that much of this shortcutting can be done implicitly (in one form or the other), without requiring explicit processing

这节课一句话

这节课把 syntax 从“旧时代的 NLP 脚手架”重新讲成一个结构化预测范例：我们先用树把句子中的远距离关系拉近，再用 PCFG、tree linearisation、span scoring 和 CYK 说明 parser 如何从局部分数拼出一棵全局合法的 tree。

1. 为什么还要讲 Syntax

Syntax

- Syntactic understanding of text was needed as a scaffold for understanding meaning
- Understanding the syntactic structure of text, in a sense, “shortcuts” distances between words and phrases in (otherwise sequential) text, providing direct relationship between them that can be further processed
- LLMs taught us that much of this shortcutting can be done implicitly (in one form or the other), without requiring explicit processing

人话直觉

想象一句话是一条很长的绳子，syntax 就是在绳子上打结：本来相隔很远的词，一旦在树上成为父子或同一个短语里的成员，就像被一根结构线拉到了一起。LLM 可以隐式学到很多这种线，但 parsing 让这根线变成可见、可检查、可约束的对象。

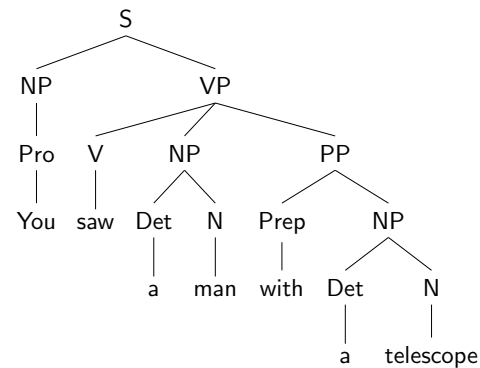
精确定义 / 机制： Syntax describes how words combine into phrases and sentences. 在传统 NLP 中，句法结构被看作通往 meaning 的 scaffold：先知道句子结构，再进一步做语义解释。Lecture 19 特别强调 syntax 的 shortcut 作用：顺序文本里两个词之间可能隔了很多 token，但 syntactic tree 能直接表示它们的结构关系。现代 LLM 不一定显式调用 parser，但 parsing 仍然是 structured prediction 的经典案例，也仍然服务于低资源多语、constrained decoding、synthetic data 和认知比较。

考试问法：如果考“为什么 LLM 时代还研究 parsing”，不要只答“为了更准确”。应答出两层：第一，syntax 把结构和远距离关系显式化；第二，parsing 提供 structured prediction、可解释性、低资源/多语和受约束输出的工具箱。

2. 两种树: Constituency 与 Dependency

To ground things

Phrase-structure trees:



人话直觉

Constituency 像收纳盒: You saw a man with a telescope 先把 a telescope 装成 NP, 再和 with 组成 PP, 再挂到更大的短语里。Dependency 更像组织架构图: 每个词找自己的老大, 边表示谁支配谁。

精确定义 / 机制: Phrase-structure 或 constituency parsing 的节点包括非终结符类别, 如 S、NP、VP、PP, 叶子是 words/tokens; 它关心连续 span 如何组成嵌套 constituents。Dependency parsing 的节点通常是词, 边表示 head-dependent relation, 边还可以标注 subj、obj 等关系。两者的关注点不同: constituency 强调短语边界、层级和 span; dependency 强调词间语法关系, 尤其适合词序较自由的语言, 因为 head-dependent 关系可比固定线性位置更稳定。

考试问法: 常见对比题: phrase-structure vs dependency。答题模板是: 前者表示短语层级和 constituents, 适合讨论 span 与嵌套结构; 后者表示词与词的 head-dependent 关系, 适合关系抽取、跨语言尤其 free word order 场景。

3. Grammar: 有限规则如何生成无限结构

Example grammar

$S \rightarrow NP VP$	[.80]	$Det \rightarrow the$	[.10]
$S \rightarrow Aux NP VP$	[.15]	$Det \rightarrow a$	[.90]
$S \rightarrow VP$	[.05]	$Noun \rightarrow book$	[.10]
$NP \rightarrow Pronoun$	[.35]	$Noun \rightarrow flight$	[.30]
$NP \rightarrow Proper-Noun$	[.30]	$Noun \rightarrow dinner$	[.60]
$NP \rightarrow Det Nominal$	[.20]	$Proper-Noun \rightarrow Houston$	[.60]
$NP \rightarrow Nominal$	[.15]	$Proper-Noun \rightarrow NWA$	[.40]
$Nominal \rightarrow Noun$	[.75]	$Aux \rightarrow does$	[.60]
$Nominal \rightarrow Nominal Noun$	[.05]	$Aux \rightarrow can$	[.40]
$VP \rightarrow Verb$	[.35]	$Verb \rightarrow book$	[.30]
$VP \rightarrow Verb NP$	[.20]	$Verb \rightarrow include$	[.30]
$VP \rightarrow Verb NP PP$	[.10]	$Verb \rightarrow prefer$	[.20]
$VP \rightarrow Verb PP$	[.15]	$Verb \rightarrow sleep$	[.20]

Food for thought: Why does it satisfy the two desiderata?
Recursiveness (yet finite grammar), interpretation of the categories

人话直觉

Grammar 不是背一本“所有合法句子大全”，那本书永远写不完。它更像乐高说明书：零件有限、拼法递归，所以可以拼出没见过但仍然像英语的句子。

精确定义 / 机制：Chomsky 提出的 desiderata 包括：grammar 应是语言 strings 的 finite specification，而不是句子列表；它还应 revealing，即让 strings 能系统地关联到 semantics。讲义又补充一条：grammar 应捕捉 structural and distributional properties，例如 head 的位置、问句变换、哪些 phrases 可以移动。CFG rule 形如 $A \rightarrow \beta$ ；PCFG 在此基础上给每条 rule 赋概率。同一左侧 nonterminal 的所有展开概率通常归一化，一棵 parse tree 的概率可由树中使用的 rule probabilities 相乘得到。

$$P(T) = \prod_{r \in T} P(r), \quad \sum_{\beta} P(A \rightarrow \beta) = 1$$

考试问法：若考 PCFG，必须同时说 rule、probability 和 tree probability：PCFG = CFG + rule probabilities；同一 LHS 展开概率和为 1；树概率是所有规则概率的乘积；parser 常寻找概率最高的 tree。

4. Parser Learning: 从规则到 Treebank

Parser learning

Parser: given a sentence, output its corresponding syntactic tree (phrase structure)

Two main schemes:

- Unsupervised parsing - learning only from text to create a parser
- Supervised parsing - learning from annotated data (such as a treebank) to create a parser

We will focus on the latter. While there is still work about doing neural unsupervised parsing, this problem is less in the focus in the current cycle of NLP. In English, most work uses the Penn Treebank ([Marcus et al., 1993](#))

人话直觉

Parser 的工作像老师批改句子结构题：给它一句话，它不只判断“这句话像不像英语”，而是要画出这句话的骨架。学习这个技能可以靠老师给标准答案，也可以自己从大量句子里摸索，但后者非常难。

精确定义 / 机制： Parser is a model that maps a sentence to its corresponding syntactic tree. 讲义区分两条学习路线：unsupervised parsing 只从 raw text 学 parser 或 grammar；supervised parsing 使用 treebank，也就是带人工 parse tree 标注的语料。当前课程重点放在 supervised parsing，英语中经典资源是 Penn Treebank。需要注意 grammar learning 和 parsing 不完全等价：前者学习规则或结构分布，后者是对一个具体 sentence 输出 tree。

考试问法： 考 supervised vs unsupervised 时，核心不是背优缺点清单，而是说清监督信号：supervised 有 gold parse trees，目标明确但标注昂贵、领域和语言覆盖有限；unsupervised 不需要树标注但搜索空间巨大，学到的结构未必符合人类 treebank 约定。

5. Grammar as a Foreign Language

Early “neural” parsing

“Grammar as a foreign language” (Vinyals et al., 2015):

- Simply use a seq2seq model (LSTM) to take as input a sentence and return its corresponding tree
- Learn based on a treebank (Penn Treebank, for example)
- **Food for thought:** But a tree is a nested structure. And seq2seq returns sequences. What do we do? Linearisation of the tree:

(S (NP (Pro You)) (VP (V saw) (NP (Det a) (N man) (PP (P with) (Det a) (N telescope)))))

We can simplify it by stripping out the words:

(S (NP Pro) (VP V (NP Det N (PP P Det N))))

And we can make it easier to process by annotating also the closing brackets:

(S (NP Pro)_{NP} (VP V (NP Det N (PP P Det N)_{PP})_{NP})_{VP})_S

人话直觉

这一步很像把一棵圣诞树压平成一串括号：树本来是二维嵌套结构，seq2seq 只能吐一维序列，那就把每次打开和关闭短语的动作都写成 token。

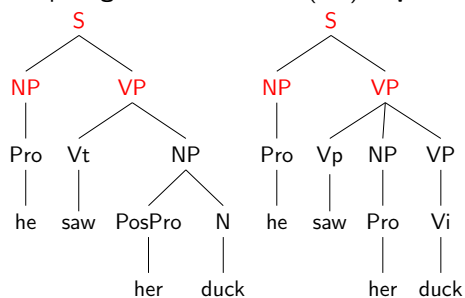
精确定义 / 机制： Vinyals et al. (2015) 的思路是用 LSTM seq2seq：输入 sentence，输出 linearised tree。树可以写成类似 (S (NP Pro) (VP V ...)) 的括号序列；为了让模型更容易恢复结构，还可在 closing brackets 上标注对应类别，如)NP、)VP、)S。这个方法几乎 grammar-less：模型不显式使用 CFG 规则，而是从 treebank 学 sentence-to-tree-string 映射。

考试问法：如果考 tree linearisation 的价值与风险：价值是把结构预测转化为 sequence generation，方便使用神经 seq2seq；风险是输出可能括号不匹配或不是合法 tree，需要 posthoc repair，而且全局结构约束不如 chart parsing 强。

6. 怎么评测一棵树

Evaluating parse accuracy

Compare **gold standard** tree (left) to **parser output** (right):



- **Precision:** ($\#$ correct constituents) / ($\#$ in parser output) = $3/5$
- **Recall:** ($\#$ correct constituents) / ($\#$ in gold standard) = $3/4$
- **F-score:** balances precision/recall: $2pr/(p+r)$

人话直觉

如果学生画的树和标准答案不完全一样，不能立刻给 0 分；我们会看他画对了多少个短语盒子，也看标准答案里的盒子被找回了多少。

精确定义 / 机制： Constituent evaluation 比较 predicted tree 和 gold tree 中的 constituents。一个 predicted constituent 若覆盖的 sentence positions 与某个 gold constituent 相同，就可计为 correct；更严格的 labelled evaluation 还要求 constituent labels 匹配。Precision 是预测出的 constituents 中有多少正确，recall 是 gold constituents 中有多少被预测找回，F-score/F1 用调和平均平衡二者。Pre-terminals 通常不计为 constituents。

$$\text{Precision} = \frac{|C_{pred} \cap C_{gold}|}{|C_{pred}|}, \quad \text{Recall} = \frac{|C_{pred} \cap C_{gold}|}{|C_{gold}|}, \quad F_1 = \frac{2PR}{P+R}$$

考试问法： 考试若给 gold tree 和 parser output，让你算 F1：先列出 predicted constituents 与 gold constituents，数 correct；再算 precision、recall、F1。不要把 POS pre-terminals 算进去，也要注意题目是否要求 labelled match。

7. Transformer Parser: 给所有 Span 打分

Parsing with Transformers

The **decoder** of **Kitaev and Klein (2018)** works as follows:
A real-valued score $s(T)$ is assigned to a tree T :

$$s(T) = \sum_{(i,j,\ell) \in T} s(i,j,\ell)$$

where $s(i,j,\ell)$ is a score for the constituent located between position i and position j with label ℓ .

At test time, we compute:

$$\hat{T} = \operatorname{argmax}_T s(T)$$

This can be found efficiently using CYK. **This gives us the optimal output sequence, unlike beam search!**

人话直觉

神经 parser 不再先拿一张显式 grammar 表逐条套规则，而像给句子里所有可能的区间贴便利贴：从第 i 个词到第 j 个词能不能是 NP？能不能是 VP？每个选择都有一个分数。

精确定义 / 机制： Kitaev and Klein (2018) 使用 Transformer encoder 得到每个 input word 的 context-aware summary vector，包含 word、POS tag 和 position information。Decoder 给每个 constituent span (i,j) 和 label ℓ 一个实值分数 $s(i,j,\ell)$ 。整棵 tree T 的分数是其中所有 span-label scores 的总和；测试时选择分数最高的合法 tree。关键点是 Transformer 本身只提供表示和局部分数，不自动保证输出是一棵合法 parse tree，因此需要 structured decoding。模型还尝试 factored attention，把 content information 与 position information 分离，因为 constituency parsing 特别依赖连续 span 的边界、距离和位置。

$$s(T) = \sum_{(i,j,\ell) \in T} s(i,j,\ell), \quad \hat{T} = \arg \max_T s(T)$$

考试问法：若考 Transformer-based constituency parser，标准答案应包含三步：Transformer 编码句子；为所有可能 span-label 组合打分；用 chart/CYK-style dynamic programming 在合法树空间中找总分最高的 tree。若追问 factored attention，说明 position matters for parsing：constituents 是连续 span，边界和距离非常关键。

8. CYK: 为什么局部高分不等于好树

Dynamic programming

Break a problem into sub-problems of the same type, solve them (by breaking them too to even smaller sub-problems, ...) and then combine their solution into a solution for the current level

In the case of parsing, we hold a data structure called a chart $A(i, j, \ell)$ which will hold the score of the “best” tree that spans indices i through j with label ℓ at the top.

$$A(i, j, \ell) = \max_{k, \ell_1, \ell_2} A(i, k, \ell_1) + A(k, j, \ell_2) + s(i, j, \ell)$$

k ranges over mid-points between i and j

The idea: The best tree for the current level is going to be made from some two other best sub-trees for some midpoint

Food for thought: Why is this the case? For a given tree, if we were not using the best sub-trees, we could make that tree achieve a higher score

Food for thought: How do we get a tree from that? Backpointers

Shay Cohen

ATNLP Lecture 19

18/26

原 PPT 第 24 页

人话直觉

你不能只挑每个局部最高分的短语，因为几个看似漂亮的 span 可能互相交叉，拼不成一棵树。CYK 像搭积木比赛：每个大积木都必须由左边一块和右边一块合法小积木拼出来。

精确定义 / 机制： Dynamic programming 把 parsing 拆成同类型子问题。Chart $A(i, j, l)$ 存储 span i through j 、顶部 label 为 l 的最佳子树分数。对每个 span，枚举 midpoint k 和左右子树标签 l_1 、 l_2 ，把左子树最佳分数、右子树最佳分数和当前 span score 相加，取最大值。最优子结构成立的原因是：如果某棵最优大树内部用了非最优子树，把它替换成同边界同标签的更优子树会让整棵树分数更高，矛盾。Backpointers 记录最佳 k 、 l_1 、 l_2 ，用于从 chart 恢复完整 tree。

$$A(i, j, \ell) = \max_{k, \ell_1, \ell_2} [A(i, k, \ell_1) + A(k, j, \ell_2) + s(i, j, \ell)]$$

考试问法：若考 why dynamic programming/CYK，答题要点是 tree 空间指数级、span 子问题重叠、最优树有最优子结构、chart 存分数、backpointers 恢复树。还要说它给的是全局最优合法 tree，不是 beam search 的近似序列。

考试速记版

- Syntax 的作用：显式表示词和短语如何组合，并 shortcut 线性距离中的长距离关系。
- LLM 时代仍学 parsing：它是 structured prediction 的经典例子，也有低资源、多语、受约束输出和可解释价值。
- Constituency tree：节点是 S/NP/VP/PP 等短语类别，核心是 nested constituents 和 spans。
- Dependency tree：节点通常是词，边表示 head-dependent relation，可带 subj/obj 等 label。
- 好的 grammar：有限 specification、能关联 meaning、捕捉 structural/distributional properties。

- PCFG: 给 CFG rules 加概率; 同一 LHS 的展开概率和为 1; tree probability 是规则概率乘积。
- Parser learning: supervised parsing 依赖 treebank; unsupervised parsing 只从 text 学结构但更难。
- Grammar as a foreign language: 把 tree linearise 成 bracket sequence, 用 seq2seq 从句子生成树串。
- Parsing F1: precision 看预测 constituents 中多少正确, recall 看 gold constituents 中多少被找回。
- Transformer parser: 编码 words, 给每个 span-label 打分, 再用 CYK/chart decoding 找最高分合法 tree。
- CYK 关键: chart 存子 span 最优解, 枚举 split 合并, backpointers 恢复 tree。
- Factored attention: 把 content 和 position 分开建模, 因为 parsing 对 span 边界和位置特别敏感。

一段稳妥考场回答

A modern constituency parser can be described as a structured prediction system. Given a sentence, a Transformer encoder first produces context-aware representations for each word, incorporating lexical, tag and positional information. The parser then assigns scores to possible labelled spans, where $s(i,j,l)$ measures how plausible it is that the words from i to j form a constituent with label l . A complete tree receives the sum of the scores of all its constituents, so decoding requires finding the highest-scoring legal tree, not merely choosing locally high-scoring spans. This is why chart parsing or CYK-style dynamic programming is used: the chart stores the best subtree for each span and label, enumerates split points, combines optimal subtrees, and records backpointers to recover the final tree. Compared with older PCFG-style parsing, the neural model learns richer contextual span scores, but it still needs explicit global decoding to guarantee a valid parse tree.